STUDENT MANAGEMENT SYSTEM

PROJECT DOCUMENTATION

A Python-Based Student Record Management Application

Submitted by

Priyanshi Dubey

Registration no.: 26BAI10873

VIT Bhopal University

Academic Project Documentation

TABLE OF CONTENTS

S. No.	Chapter / Section	Page
1	Introduction	6
2	Problem Statement	6
3	Objectives	6
4	Scope of the Project	7
5	Target Users	7
6	Functional Requirements	8
7	Non-Functional Requirements	9
8	System Design and Methodology	10
9	Implementation Details	13
10	Testing and Results	27
11	Challenges Faced	32
12	Learning Outcomes	32
13	Limitations	32
14	Future Enhancements	33
15	Conclusion	33
16	References	34

1. Introduction

The Student Management System is a Python-based application developed to simplify and organize the management of student information. Educational institutions handle a large amount of student data, including names, roll numbers, courses, contact details, and other academic information. Managing such information manually can be time-consuming and may result in errors or loss of records.

The proposed system provides a simple and efficient way to perform common student management operations through a computer-based application. The system can support operations such as adding new student records, viewing existing records, searching for students, updating information, and deleting records when required.

The project demonstrates practical implementation of Python programming concepts such as variables, functions, conditional statements, loops, data structures, input validation, and exception handling. It also provides a foundation for developing a more advanced student information management platform.

2. Problem Statement

Managing student records manually becomes difficult as the number of students increases. Traditional methods such as notebooks, registers, or manually maintained files require considerable effort and may lead to duplicate, outdated, or incorrect information.

- Difficulty in maintaining a large number of student records.
- Time-consuming searching and retrieval of records.
- Difficulty updating student information.
- Possibility of human errors and incorrect entries.
- Repetitive manual work.
- Difficulty maintaining consistent and organized records.
- Difficulty retrieving information quickly.

The Student Management System addresses these issues by providing a centralized application for performing basic operations on student records.

3. Objectives

1. To create a computerized system for managing student records.
2. To provide an easy method for adding new students.
3. To display stored student information.
4. To provide a search facility for finding student records.
5. To allow modification of existing student details.
6. To provide an option for deleting records when required.
7. To reduce manual effort involved in maintaining student information.
8. To minimize errors in record management.
9. To demonstrate practical implementation of Python programming concepts.
10. To create a foundation that can be extended into a larger student management application.

4. Scope of the Project

The scope of the Student Management System covers the basic operations required for maintaining student records. The system focuses on record creation, viewing, searching, updating, and deletion.

- Adding student information.

- Viewing student records.
- Searching for students.
- Updating student details.
- Deleting student records.
- Validating user input.
- Displaying appropriate success and error messages.

5. Target Users

Target User	Possible Use
Teachers	Maintain and quickly search basic student information.
Administrative Staff	Add, update, search, and manage student records.
Educational Institutions	Use as a basic student record-management solution.
Students	Benefit from organized maintenance of their information.
Learners / Developers	Study Python programming and CRUD-style applications.

6. Functional Requirements

Requirement	Description
Add Student	Allows the user to enter and store student information.
View Students	Displays available student records.
Search Student	Finds a student using an identifier.
Update Student	Modifies information of an existing student.
Delete Student	Removes a student record.
Input Validation	Checks whether entered information is valid.
Exit	Allows the user to safely close the application.

6.1 Add Student

The user enters the required student information and the system creates a new record.

6.2 View Student Records

The system displays stored student information in a readable format.

6.3 Search Student

The search function allows the user to locate a student using an appropriate identifier.

6.4 Update Student

The update operation modifies information belonging to an existing student.

6.5 Delete Student

The delete operation removes a selected student record after identifying the appropriate record.

7. Non-Functional Requirements

Requirement	Description
Usability	Simple interface suitable for users with basic computer knowledge.
Performance	Basic operations should execute without unnecessary delay.
Reliability	The application should behave consistently during normal operation.
Maintainability	Functions and logical structure should make future changes easier.
Portability	Python allows execution on systems with a compatible environment.
Security	Input validation should reduce unintended or invalid operations.

8. System Design and Methodology

8.1 System Workflow

The application follows an input-processing-output workflow.

8.2 Input-Process-Output Model

Stage	Description
Input	Student details and selected operation.
Processing	Python program processes the requested operation.
Storage	Student information is maintained using the selected storage method.
Output	Records or status messages are displayed to the user.

8.3 Development Methodology

11. Requirement Analysis – identify required student management operations.
12. System Design – plan menu structure, records, and operations.
13. Implementation – develop the system using Python.
14. Testing – test each operation with different input cases.
15. Debugging – identify and correct errors.
16. Finalization – perform final verification after corrections.

9. Implementation Details

The Student Management System is implemented using Python. The implementation uses fundamental programming constructs to organize student data and perform management operations.

9.1 Technologies Used

Technology / Concept	Purpose
Python	Main programming language.
Functions	Organize system operations.
Conditional Statements	Decision making.
Loops	Repeated menu execution.
Lists / Dictionaries	Managing student information.
File Handling	Persistent storage where implemented.
Exception Handling	Handling invalid input and runtime errors.

9.2 Main Menu

A typical menu structure is shown below:

```
===== STUDENT MANAGEMENT SYSTEM =====

1. Add Student
2. View Students
3. Search Student
4. Update Student
5. Delete Student
6. Exit

Enter your choice:
```

9.3 Add Student Module

The Add Student module collects the required student information and creates a new record.

9.4 View Student Module

The View module retrieves available student records and displays them to the user.

9.5 Search Module

The Search module locates a particular student based on a suitable identifier.

9.6 Update Module

The Update module modifies existing information after locating the relevant student record.

9.7 Delete Module

The Delete module removes a selected student record.

9.8 Error Handling

Error handling is important because users may enter unexpected values. Examples include invalid menu choices, incorrect numerical input, empty fields, or searches for nonexistent records. Python's try-except mechanism can be used to handle runtime input errors.

```python
import json
import os
FILE_NAME = "students.json"
# ================== FILE HANDLING ====================

def load_students():
    """Load student data from JSON file."""
    if os.path.exists(FILE_NAME):
        try:
            with open(FILE_NAME, "r") as file:
                return json.load(file)
        except (json.JSONDecodeError, IOError):
            return {}

    return {}


def save_students(students):
    """Save student data to JSON file."""
    try:
        with open(FILE_NAME, "w") as file:
            json.dump(students, file, indent=4)
    except IOError:
        print("Error: Unable to save student data.")


# ================== VALIDATION ====================

def get_marks(subject):
    """Get valid marks between 0 and 100."""
    while True:
        try:
            marks = float(input(f"Enter marks for {subject}: "))
            if 0 <= marks <= 100:
                return marks

            print("Marks must be between 0 and 100.")

        except ValueError:
            print("Please enter a valid number.")


def get_grade(percentage):
    """Calculate grade based on percentage."""
    if percentage >= 90:
        return "A+"
    elif percentage >= 80:
        return "A"
    elif percentage >= 70:
        return "B"
    elif percentage >= 60:
        return "C"
    elif percentage >= 50:
        return "D"
    elif percentage >= 40:
        return "E"
    else:
        return "F"


# ================== ADD STUDENT ====================

def add_student(students):
    print("\n" + "=" * 55)
    print("                ADD STUDENT")
    print("=" * 55)

    student_id = input("Enter Student ID: ").strip()

    if not student_id:
        print("Student ID cannot be empty.")
        return

    if student_id in students:
        print("Student ID already exists.")
        return

    name = input("Enter Student Name: ").strip()
    age = input("Enter Age: ").strip()
    gender = input("Enter Gender: ").strip()
    course = input("Enter Course: ").strip()
    semester = input("Enter Semester: ").strip()
    email = input("Enter Email: ").strip()
    phone = input("Enter Phone Number: ").strip()

    print("\nEnter marks:")
    python = get_marks("Python")
    dbms = get_marks("DBMS")
    dsa = get_marks("Data Structures")
    mathematics = get_marks("Mathematics")
    computer_networks = get_marks("Computer Networks")

    total = python + dbms + dsa + mathematics + computer_networks
    percentage = total / 5
    grade = get_grade(percentage)

    students[student_id] = {
        "name": name,
        "age": age,
        "gender": gender,
        "course": course,
        "semester": semester,
        "email": email,
        "phone": phone,
        "marks": {
            "Python": python,
            "DBMS": dbms,
            "Data Structures": dsa,
            "Mathematics": mathematics,
            "Computer Networks": computer_networks
        },
        "total": total,
        "percentage": percentage,
        "grade": grade
    }

    save_students(students)

    print("\nStudent added successfully.")
    print(f"Percentage: {percentage:.2f}%")
    print(f"Grade: {grade}")


# ================== VIEW STUDENTS ====================

def view_students(students):
    print("\n" + "=" * 90)
    print("                        ALL STUDENTS")
```

```
    print("=" * 90)

    if not students:
        print("No student records found.")
        return

    print(
        f"{'ID':<10}"
        f"{'Name':<22}"
        f"{'Course':<15}"
        f"{'Semester':<10}"
        f"{'Percentage':<15}"
        f"{'Grade':<8}"
    )

    print("-" * 90)

    for student_id, student in students.items():
        print(
            f"{student_id:<10}"
            f"{student['name'][:20]:<22}"
            f"{student['course']:<15}"
            f"{student['semester']:<10}"
            f"{student['percentage']:.2f}%{'':<9}"
            f"{student['grade']:<8}"
        )

    print("-" * 90)


# ==================== VIEW STUDENT DETAILS ====================
    keyword = input("Enter Student ID or Name: ").strip().lower()

    found = False

    for student_id, student in students.items():

        if (
            keyword in student_id.lower()
            or keyword in student["name"].lower()
        ):
            display_student(student_id, student)
            found = True

    if not found:
        print("No matching student found.")


# ==================== UPDATE STUDENT ====================

def update_student(students):
    print("\n" + "=" * 55)
    print("                UPDATE STUDENT")
    print("=" * 55)

    student_id = input("Enter Student ID: ").strip()

    if student_id not in students:
        print("Student not found.")
        return

    student = students[student_id]
```

```
def display_student(student_id, student):
    print("\n" + "=" * 55)
    print("                  STUDENT DETAILS")
    print("=" * 55)

    print(f"Student ID : {student_id}")
    print(f"Name       : {student['name']}")
    print(f"Age        : {student['age']}")
    print(f"Gender     : {student['gender']}")
    print(f"Course     : {student['course']}")
    print(f"Semester   : {student['semester']}")
    print(f"Email      : {student['email']}")
    print(f"Phone      : {student['phone']}")

    print("\nMarks")
    print("-" * 40)

    for subject, marks in student["marks"].items():
        print(f"{subject:<25}: {marks}")

    print("-" * 40)
    print(f"Total      : {student['total']:.2f}")
    print(f"Percentage : {student['percentage']:.2f}%")
    print(f"Grade      : {student['grade']}")


# ==================== SEARCH STUDENT ====================

def search_student(students):
    print("\n" + "=" * 55)
    print("                SEARCH STUDENT")
    print("=" * 55)
```

```
    print("\nPress Enter to keep the current value.")

    name = input(f"Name [{student['name']}]: ").strip()
    age = input(f"Age [{student['age']}]: ").strip()
    gender = input(f"Gender [{student['gender']}]: ").strip()
    course = input(f"Course [{student['course']}]: ").strip()
    semester = input(f"Semester [{student['semester']}]: ").strip()
    email = input(f"Email [{student['email']}]: ").strip()
    phone = input(f"Phone [{student['phone']}]: ").strip()

    if name:
        student["name"] = name

    if age:
        student["age"] = age

    if gender:
        student["gender"] = gender

    if course:
        student["course"] = course

    if semester:
        student["semester"] = semester

    if email:
        student["email"] = email

    if phone:
        student["phone"] = phone
```

```
    save_students(students)

    print("\nStudent information updated successfully.")


# ================== DELETE STUDENT ==================

def delete_student(students):
    print("\n" + "=" * 55)
    print("                DELETE STUDENT")
    print("=" * 55)

    student_id = input("Enter Student ID: ").strip()

    if student_id not in students:
        print("Student not found.")
        return

    student = students[student_id]

    print(f"\nStudent: {student['name']}")

    confirm = input("Are you sure you want to delete? (Y/N): ").strip().upper()

    if confirm == "Y":
        del students[student_id]
        save_students(students)
        print("Student deleted successfully.")
    else:
        print("Deletion cancelled.")
```

```
        print("Deletion cancelled.")


# ==================== COURSE SEARCH ====================

def search_by_course(students):
    print("\n" + "=" * 55)
    print("                SEARCH BY COURSE")
    print("=" * 55)

    course = input("Enter Course: ").strip().lower()

    found = False

    print("\nStudents enrolled in", course.upper())
    print("-" * 65)

    for student_id, student in students.items():

        if student["course"].lower() == course:

            print(
                f"ID: {student_id} | "
                f"Name: {student['name']} | "
                f"Semester: {student['semester']}"
            )

            found = True

    if not found:
        print("No students found for this course.")
```

```
# ==================== STATISTICS ====================

def statistics(students):
    print("\n" + "=" * 55)
    print("                STUDENT STATISTICS")
    print("=" * 55)

    if not students:
        print("No student records available.")
        return

    total_students = len(students)

    percentages = [
        student["percentage"]
        for student in students.values()
    ]

    average = sum(percentages) / total_students

    highest_id = max(
        students,
        key=lambda x: students[x]["percentage"]
    )

    lowest_id = min(
        students,
        key=lambda x: students[x]["percentage"]
    )

    passed = sum(
        1 for student in students.values()
        if student["percentage"] >= 40
    )
    failed = total_students - passed

    print(f"Total Students : {total_students}")
    print(f"Average Score  : {average:.2f}%")
    print(f"Passed Students: {passed}")
    print(f"Failed Students: {failed}")

    print(
        f"\nHighest Score  : "
        f"{students[highest_id]['name']} "
        f"({students[highest_id]['percentage']:.2f}%)"
    )

    print(
        f"Lowest Score   : "
        f"{students[lowest_id]['name']} "
        f"({students[lowest_id]['percentage']:.2f}%)"
    )


# ==================== MAIN MENU ====================

def main():

    students = load_students()

    while True:

        print("\n")
        print("=" * 60)
        print("          STUDENT MANAGEMENT SYSTEM")
        print("=" * 60)
```

```
        print("1.  Add Student")
        print("2.  View All Students")
        print("3.  Search Student")
        print("4.  Update Student")
        print("5.  Delete Student")
        print("6.  Search Students by Course")
        print("7.  Student Statistics")
        print("8.  Exit")

        print("=" * 60)

        choice = input("Enter your choice: ").strip()

        if choice == "1":
            add_student(students)

        elif choice == "2":
            view_students(students)

        elif choice == "3":
            search_student(students)

        elif choice == "4":
            update_student(students)

        elif choice == "5":
            delete_student(students)

        elif choice == "6":
            search_by_course(students)

        elif choice == "7":
            statistics(students)

        elif choice == "8":
            print("\nThank you for using Student Management System.")
            break

        else:
            print("\nInvalid choice. Please enter a number from 1 to 8.")


# ==================== START PROGRAM ====================

if __name__ == "__main__":
    main()
```

10. Testing and Results

Testing was performed to verify that the Student Management System performs its intended operations correctly. Each major function was considered separately and tested using valid and invalid input scenarios.

10.1 Testing Objectives

- Verify each system function.
- Identify incorrect outputs.
- Check invalid inputs.
- Verify student record operations.
- Ensure the application behaves correctly during normal usage.

10.2 Test Cases

Test ID	Test Case	Input	Expected Result	Status
TC01	Add student	Valid details	Student added	Pass
TC02	View records	View option	Records displayed	Pass
TC03	Search student	Existing ID	Student displayed	Pass
TC04	Search student	Invalid ID	Not-found message	Pass

TC05	Update student	Existing ID	Record updated	Pass
TC06	Delete student	Existing ID	Record deleted	Pass
TC07	Invalid menu	Invalid option	Error message	Pass
TC08	Exit	Exit option	Program terminates	Pass

10.3 Final Testing Result

The testing process verifies the major operations of the Student Management System. Valid inputs are expected to produce the intended results, while invalid inputs should be handled through appropriate messages or validation mechanisms. The final results should be updated according to the actual execution of the submitted program

```
============================================================
           STUDENT MANAGEMENT SYSTEM
============================================================
1.  Add Student
2.  View All Students
3.  Search Student
4.  Update Student
5.  Delete Student
6.  Search Students by Course
7.  Student Statistics
8.  Exit
============================================================
Enter your choice: 1

====================================================
                ADD STUDENT
====================================================
Enter Student ID: 1245
Enter Student Name: tani
Enter Age: 18
Enter Gender: f
Enter Course: cse core
Enter Semester: 1
Enter Email: mjgui
Enter Phone Number: 32569874

Enter marks:
Enter marks for Python: 85
Enter marks for DBMS: 95
Enter marks for Data Structures: 87
Enter marks for Mathematics: 97
Enter marks for Computer Networks: 98

Student added successfully.
```

```
5.  Delete Student
6.  Search Students by Course
7.  Student Statistics
8.  Exit
============================================================
Enter your choice: 3

====================================================
                SEARCH STUDENT
====================================================
Enter Student ID or Name: tani

====================================================
                STUDENT DETAILS
====================================================
Student ID : 1245
Name       : tani
Age        : 18
Gender     : f
Course     : cse core
Semester   : 1
Email      : mjgui
Phone      : 32569874

Marks
----------------------------------------
Python                  : 85.0
DBMS                    : 95.0
Data Structures         : 87.0
Mathematics             : 97.0
Computer Networks       : 98.0
----------------------------------------
Total      : 462.00
Percentage : 92.40%
```

```
2.  View All Students
3.  Search Student
4.  Update Student
5.  Delete Student
6.  Search Students by Course
7.  Student Statistics
8.  Exit
====================================================
Enter your choice: 2

================================================================================
                        ALL STUDENTS
================================================================================
ID        Name                Course         Semester  Percentage     Grade
--------------------------------------------------------------------------------
2154      fffv                bg             1         22.60%         F
26bai10463vaishnavi           cs aiml        2         56.00%         D
1245      tani                cse core       1         92.40%         A+
--------------------------------------------------------------------------------


====================================================
          STUDENT MANAGEMENT SYSTEM
====================================================
1.  Add Student
2.  View All Students
3.  Search Student
4.  Update Student
5.  Delete Student
6.  Search Students by Course
7.  Student Statistics
8.  Exit
====================================================
```

11. Challenges Faced

- Managing multiple student records in an organized structure.
- Validating different types of user input.
- Implementing reliable search functionality.
- Updating and deleting the correct student record.
- Handling unexpected inputs without terminating the program.
- Debugging logical and runtime errors.

12. Learning Outcomes

- Understanding Python programming fundamentals.
- Using functions and modular programming.
- Applying conditional statements and loops.
- Working with lists and dictionaries.
- Handling input and output.
- Using file handling where applicable.
- Applying exception handling.
- Understanding CRUD operations.
- Performing testing and debugging.
- Preparing technical project documentation.

13. Limitations

17. The system may have limited user authentication.
18. It may not support multiple users simultaneously.
19. Advanced database management may not be implemented.
20. The interface may be command-line based.
21. The system may not provide online access.
22. Advanced reporting may not be available.
23. Backup and recovery features may be limited.
24. Attendance, fees, examinations, and timetable modules may not be included.

14. Future Enhancements

- Develop a graphical user interface using Tkinter, PyQt, or another suitable framework.
- Integrate a database such as SQLite, MySQL, or PostgreSQL.
- Add authentication for authorized users.
- Add attendance management.
- Add examination and result management.
- Add fee management.
- Generate PDF or Excel reports.
- Convert the application into a web-based system.

15. Conclusion

The Student Management System provides a simple and structured approach to managing student information. The project demonstrates how Python programming can be used to create an application capable of performing common student record operations.

The system reduces the effort involved in manually managing records and provides a convenient method for adding, viewing, searching, updating, and deleting student information. The project also provides practical experience in programming, data management, validation, error handling, testing, debugging, and technical documentation.

Although the current system focuses on basic student management, its structure can be extended with database integration, authentication, attendance, examination management, fee management, report generation, and graphical or web-based interfaces.

16. References

25. Python Software Foundation – Python Documentation: https://docs.python.org/3/
26. Python Software Foundation – Python Tutorial: https://docs.python.org/3/tutorial/
27. W3Schools – Python Tutorial: https://www.w3schools.com/python/
28. GeeksforGeeks – Python Programming Language: https://www.geeksforgeeks.org/python-programming-language/
29. Real Python – Python Programming Tutorials: https://realpython.com/